Nav2, Start to Finish

Contents

1. Overview . . . 2
2. Bring-up: launch files and the lifecycle manager . . . 3
   - 2.1 Why the start-up needs an order . . . 3
   - 2.2 The launch files . . . 4
   - 2.3 The lifecycle manager . . . 4
3. One goal, message by message . . . 5
4. Where the robot is: the map server and AMCL . . . 7
   - 4.1 The map, and why odometry is not enough . . . 7
   - 4.2 AMCL . . . 7
   - 4.3 The correction AMCL publishes . . . 8
5. Where the robot may go: the global and local costmaps . . . 9
   - 5.1 Why two costmaps, built from layers . . . 10
   - 5.2 The inflation cost . . . 11
   - 5.3 Global and local . . . 11
6. The behaviour tree and the planner server . . . 11
   - 6.1 The behaviour tree navigator . . . 11
   - 6.2 The planner server and NavFn . . . 12
7. The controller server . . . 13
   - 7.1 One controller cycle . . . 14
   - 7.2 DWB in our run . . . 14
   - 7.3 Which controller: DWB, MPPI, regulated pure pursuit, TEB . . . 14
8. After the controller: the velocity smoother and the collision monitor . . . 15
   - 8.1 The velocity smoother . . . 15
   - 8.2 The collision monitor . . . 16
   - 8.3 Stop, slowdown and limit zones . . . 18
9. The goal from start to finish . . . 19
10. ROS 1 `move_base` compared . . . 21
11. Simulators and tools . . . 23
12. Summary . . . 25
13. Sources . . . 26
14. Key terms . . . 27

Where this fits:

- **Builds on:** Maps and landmarks; Likelihood field model; Particle filter; Dijkstra's algorithm; Rate limits and deadband; Layered autonomy stack and its rates; Lifecycle nodes; Parameters and launch files; DWB controller, start to finish; Behaviour trees and the tick; Goal checker and progress checker.

1. Overview

Key point: Nav2 drives a robot to a goal pose with 17 separate programs, most of them servers with one job each: a lifecycle manager starts them in order, AMCL says where the robot is, two costmaps say where it may go, a behaviour tree asks the planner for a path and the controller for commands, and a velocity smoother and a collision monitor check each command before it reaches the wheels. In our own Python version of these servers, run on the room, one goal crosses all of them and the robot reaches the door in 11.50 s, replanning once round the person.

Every part of a navigation stack has been built in this course, one at a time:

- a global path on a grid, by Dijkstra's algorithm (G-2565) and A* (G-2577);
- where the robot is, by a particle filter (G-2492) on a map;
- the commands round a person, by the local planners DWB (G-3255) and TEB;
- which step comes next, by a behaviour tree;
- and the communication between programs, by the nodes, topics and actions of ROS 2 (G-3221).

What is still missing is the complete system: which program runs each part, in what order they start, what each one sends to which, and how a goal selected on a map becomes wheel speeds. A real robot runs such a system, and the autonomy stack (G-3210) of this chapter named its layers but not its programs.

The goal is the chapter's running example: in the room of trajectory optimisation, the robot (a disk of radius 0.25 m, each wheel at most 0.6 m/s) starts at rest at (2.0, 1.0) facing 30° and must stop at the door pose (3.6, 3.5) facing 90°. At 2.4 s a person steps onto the route at (2.90, 2.75); the map does not contain them. The timed-elastic-band planner of trajectory optimisation reached the door in 8.0 s. Here the same goal goes to the navigation stack of ROS 2, **Nav2** (G-3340): a set of separate programs, each a server with one job, that together take a goal pose and drive the robot there (Nav2 docs, "Navigation Concepts"; Macenski et al. 2023 §I).

What we run. The run in this Note is **our own Python version of Nav2's servers**, written from their code on GitHub (`ros-navigation/navigation2`, branch `main`, read 2026-10-12) and the bring-up parameters in `nav2_bringup/params/nav2_params.yaml`. The run is not Nav2 itself. Every number comes from this Note's notebook, `RO-080-nav2-start-to-finish.ipynb`, and its module `images/nav2sim.py`; the controller is the Python version of DWB written for DWA and DWB (`images/dwbkit.py`, copied). What we simplified:

- all servers run in one program, in a fixed order inside each 0.05 s cycle, and every service and action answers within that cycle, with no message delays;
- the laser is two-dimensional and ray-cast on the exact room, so the local costmap's 3D voxel layer becomes a 2D obstacle layer;
- NavFn spreads its potential with a priority queue in exact order instead of its threshold buckets, over the whole map;
- AMCL's estimate is the weighted mean of all its particles (Nav2 takes the mean of the heaviest cluster);
- the collision monitor uses the latest scan as it is, without shifting it by the robot's motion since;
- the costmap filters (keep-out and speed zones) and every server our goal never calls (waypoint follower, docking, route) are left out.

Our robot replaces the bring-up file's TurtleBot where they differ: a radius of 0.25 m (the file's 0.22 m), a top speed of 0.6 m/s (the smoother's 0.5 m/s in the file), a forward acceleration of 0.5 m/s^2 (the smoother's 3.0 m/s^2), a turn-rate acceleration of 3.2 rad/s^2, as in DWA and DWB (the smoother's 3.5 rad/s^2), and odometry whose left wheel is 1 percent smaller than assumed, so that odometry drifts.

The opening question: **what does each Nav2 server do, in what order and with what numbers, between "go to the door" and the robot stopping there?** We follow the goal through:

- the bring-up: launch files and the lifecycle manager (Section 2);
- the whole system, one message at a time (Section 3);
- where the robot is: the map server and AMCL (Section 4);
- where it may go: the global and local costmaps (Section 5);
- the behaviour tree navigator and the planner server (Section 6);
- the controller server (Section 7);
- the velocity smoother and the collision monitor (Section 8);
- the whole goal from start to finish (Section 9);
- ROS 1's `move_base`, and the simulators and tools round Nav2 (Section 10, Section 11).

2. Bring-up: launch files and the lifecycle manager

> **Key point:** Before the first goal, 17 managed nodes must exist, have read their parameters and be ready, in an order in which each node's inputs come first. One lifecycle manager configures all 17 in order, then activates all 17 in order, and keeps a heartbeat with each, so that the failure of one node stops all of them.

2.1 Why the start-up needs an order

The goal of the Overview needs the map before AMCL can localise on it, and AMCL's position before any costmap can place a scan on the map. If the 17 programs started at once, a server could receive its first message before the server it depends on exists, and the goal would fail unpredictably. ROS 2's lifecycle nodes (G-3236), nodes that change state

(unconfigured, inactive, active) only when told, make the order controllable, provided that one node issues the transitions.

2.2 The launch files

Nav2's bring-up is five launch files (G-3235) in `nav2_bringup/launch` (branch `main`), shown as an example of nested launch files in ROS 2 in Nav2; for our goal, the order they set is what matters:

- `localization_launch.py` starts `map_server` and `amcl`;
- `keepout_zone_launch.py` and `speed_zone_launch.py`, both on by default (`use_keepout_zones`, `use_speed_zones`), each start two servers for a costmap filter: a mask server and a filter-info server;
- `navigation_launch.py` starts the eleven navigation servers, from `controller_server` to `following_server`; Figure 1 lists them in their order;
- `bringup_launch.py` includes the other four launch files and starts one lifecycle manager, named `lifecycle_manager_nav2`; the manager's parameter `node_names` lists the localisation nodes, then the four filter servers, then the navigation servers: 17 nodes.

A costmap filter reads a mask, a second map that marks areas the robot must keep out of or must cross slowly. Our room has no such areas, so the filter servers start but our run leaves the filters out.

The launch files also rename topics so that the commands pass through two checks: `controller_server` and `behavior_server` publish on `cmd_vel_nav` instead of `cmd_vel`, `velocity_smoother` turns `cmd_vel_nav` into `cmd_vel_smoothed`, and `collision_monitor` turns `cmd_vel_smoothed` into the `cmd_vel` the base obeys (bring-up `nav2_params.yaml`, `cmd_vel_in_topic` and `cmd_vel_out_topic`).

2.3 The lifecycle manager

The node that moves a list of lifecycle nodes through their states in a fixed order, and watches them afterwards, is the **lifecycle manager** (G-3341) (`lifecycle_manager.cpp`; Macenski et al. 2023 §IX). Its `startup()` does two passes:

1. configure every node in list order: each reads its parameters and creates its publishers, services and action servers;
2. activate every node in list order: each starts publishing and accepting goals.

Shutting down runs the same transitions backwards, from the last node to the first. Figure 1 plays the start-up. Watch the column of states: every node becomes inactive (orange) before any becomes active (green), and `map_server` leads both passes.

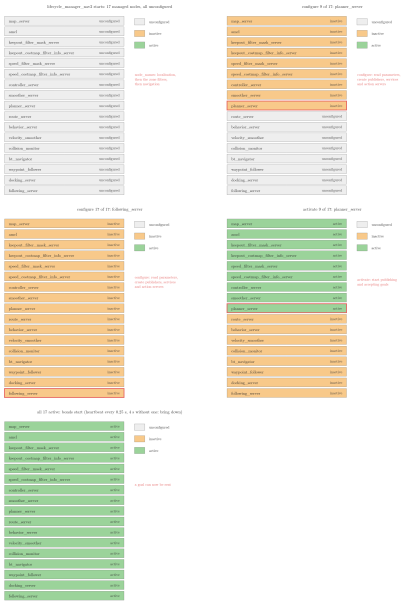


Figure 1: Animation: Nav2's bring-up, as our Python version of the lifecycle manager runs it (not Nav2 running). The 17 nodes of lifecycle_manager_nav2's node_names list, the localisation nodes (map_server, amcl), the four zone-filter servers, then the navigation nodes, all unconfigured (grey) at first. Frames 'configure 1 of 17' to 'configure 17 of 17': each node in turn becomes inactive (orange): it has read its parameters and created its publishers, services and action servers. Frames 'activate 1 of 17' to 'activate 17 of 17': each becomes active (green): it publishes and accepts goals. Last frame: all 17 active; bonds start, a heartbeat every 0.25 s, and 4 s without one brings the system down

After the second pass the manager starts a bond with each node: every node sends a heartbeat every 0.25 s, the manager checks every 0.2 s, and a node silent for 4 s is taken as failed, which brings the system down (`lifecycle_manager.cpp`: `bond_heartbeat_period` 0.25, `bond_timeout` 4.0, a 200 ms timer). In all, 17 nodes and 34 transitions (notebook). The servers are now ready, and a goal can arrive.

3. One goal, message by message

> **Key point:** The goal travels as an action to the behaviour tree navigator, which calls the planner for a path and the controller to follow it; the controller's commands then pass the velocity smoother and the collision monitor before reaching the base, while the laser, the map and AMCL feed every server.

Section 2 left 17 nodes active and waiting. Figure 2 follows the goal of the Overview through them, one message per frame, with the numbers of our run. Watch the red arrow: the goal enters at the top, the path comes back from the planner, and the commands go down the middle to the base.

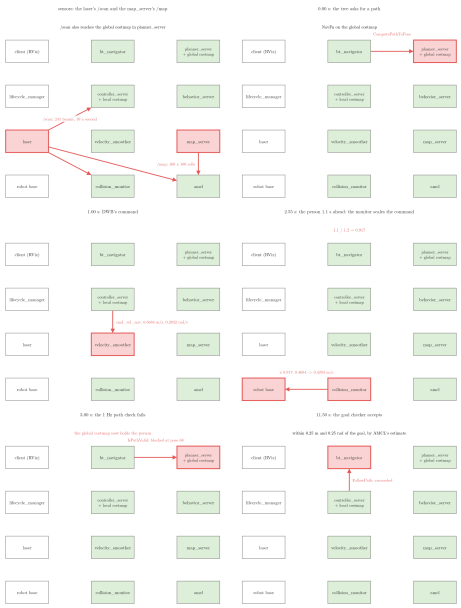


Figure 2: Animation: one goal through Nav2's servers, one message lit per frame (our Python version of Nav2's servers, main run; not Nav2 running). Boxes: client (RViz), bt_navigator, planner_server with the global costmap, lifecycle_manager, controller_server with the local costmap, behavior_server, laser, velocity_smoother, map_server, robot base, collision_monitor, amcl. Frame 'sensors': /scan, 241 beams 10 times a second, to amcl, the local costmap and the collision monitor; /map, 160 x 100 cells, to amcl. Frame 'localisation': amcl publishes map -> odom, the base /odom. 0.00 s: the NavigateToPose goal (3.6, 3.5), 90 degrees, reaches bt_navigator; ComputePathToPose to planner_server; the path, 3.26 m and 130 poses, comes back; FollowPath to controller_server. 1.00 s: cmd_vel_nav 0.5684 m/s and 0.2632 rad/s; cmd_vel_smoothed 0.525 m/s; cmd_vel 0.525 m/s. 2.55 s: the monitor scales the command by 0.917, 0.4684 to 0.4294 m/s. 3.00 s: IsPathValid finds the path blocked at pose 68; a new path, 2.986 m. 11.50 s: FollowPath succeeded; 11.51 s: the result reaches the client

Each arrow is one of the ROS 2 communication types (Nav2 docs, "Navigation Servers"):

- the goal and the two jobs it starts, ComputePathToPose and FollowPath, are actions (G-3227): long jobs with a goal, feedback and a result;
- the scan, the map, the odometry and the three velocity commands are topics (G-3223);
- the path check, IsPathValid, is a service (G-3226): one request, one reply;
- AMCL's correction is a transform from the map frame to the odometry frame (Section 4).

The planner, controller and behaviour servers each load their algorithm as a plug-in under a name, such as `FollowPath` for the controller, and host the costmap that algorithm needs, so one costmap serves every plug-in in the server (Nav2 docs, "Navigation Servers"). The next sections examine each box in the order the goal needs them, starting with the information every server needs first: where the robot is.

4. Where the robot is: the map server and AMCL

> **Key point:** AMCL is the particle filter on the map, with a likelihood field for the laser; it publishes the correction from the odometry frame to the map frame. In our run odometry drifts 0.222 m by the end, and AMCL keeps the estimate within 0.054 m, with odometry noise settings that match our robot.

4.1 The map, and why odometry is not enough

Every server of Figure 2 needs the robot's place on the map: the planner to start the path, the controller to compare the robot with the path. Odometry gives a place, but it drifts. Our robot's left wheel is 1 percent smaller than its wheel odometry (G-2321) assumes, so by the end of the drive odometry puts the robot 0.222 m and 9.3° away from where it is (notebook).

The map comes from the **map server** (G-3355), the node that loads a saved occupancy grid (G-2340) and serves it on the `/map` topic. Ours is the room in 0.05 m cells, 160 by 100 cells reaching 0.5 m beyond the walls, as a map recorded by driving through the room would.

4.2 AMCL

Nav2's localiser is **AMCL** (G-3349), adaptive Monte Carlo localisation: a particle filter (G-2492) whose particles are poses on the map, moved with the odometry motion model (G-2323) and weighted by the laser with the likelihood field model (G-2391), with a particle count that adapts to how spread out the particles are (Nav2 docs, "State Estimation"; `nav2_amcl`). Monte Carlo localisation is this particle filter run on a known map, each particle a guess of the pose; here we follow Nav2's version of it.

AMCL updates its particles only after the odometry has moved 0.25 m or turned 0.2 rad (`update_min_d`, `update_min_a`), and then:

1. moves every particle by the odometry change, with noise (`DifferentialMotionModel`);
2. scores every particle with 60 of the laser's beams against the map (`max_beams`);
3. resamples, keeping between 500 and 2000 particles (`min_particles`, `max_particles`), fewer when they are concentrated in a small region.

The score of one beam. With:

- z is the distance from where the beam ends, seen from the particle, to the nearest occupied cell of the map, in m, at most 2.0 m (`laser_likelihood_max_dist`);
- σ is the laser's noise, 0.2 m (`sigma_hit`);
- z_{hit} and z_{rand} are the weights of a true hit and of a random reading, 0.5 each in the bring-up file;
- $r_{\max}$ is the laser's range, 30 m;
- p_z is the beam's score, a number between 0 and 1.

A beam that ends on a wall, $z = 0$:

$$p_z = 0.5 + 0.5/30 = 0.5167$$

A beam that ends 0.4 m from any wall:

$$p_z = 0.5 \times e^{-0.4^2/(2\times 0.2^2)} + 0.0167$$

$$= 0.5 \times e^{-2} + 0.0167 = 0.0843$$

The same score in symbols:

$$p_z = z_{\text{hit}}\, e^{-z^2/2\sigma^2} + \frac{z_{\text{rand}}}{r_{\max}}$$

The Gaussian term is the likelihood field's density of a hit near a wall; the constant term prevents one reading from an obstacle the map lacks, such as the person, from reducing a good particle's score to almost zero. Nav2's code then adds the cubes of the 60 beam scores to 1 to weight the particle (`likelihood_field_model.cpp`): a wall hit adds 0.1379, a beam 0.4 m off adds 0.0006 (notebook).

4.3 The correction AMCL publishes

AMCL does not move the robot's odometry. AMCL publishes the transform from the map frame to the odometry frame, chosen so that the odometry pose, carried by that transform, lands on AMCL's estimate; every server then looks up the robot's place on the map through it (Nav2 docs, "State Estimation"). The three frames are those of the transform tree: map is fixed to the room, odom is where odometry started counting, and base_link moves with the robot.

Figure 3 plays AMCL's 21 updates in our run. Watch the three lines: the grey dashed odometry bends away to the right, while the blue estimate stays on the black true path.

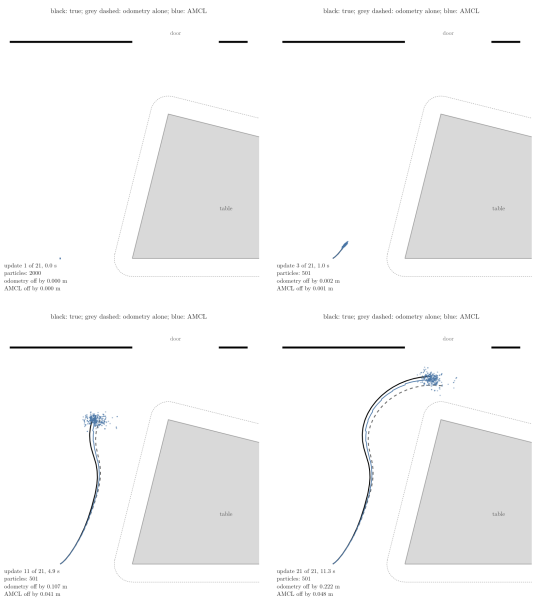


Figure 3: Animation: AMCL in the main run, one frame per filter update (our Python version of nav2_amcl; not Nav2 running). Black: where the robot truly is; grey dashed: where odometry alone puts it (left wheel 1 percent smaller than assumed); blue: AMCL's estimate, with up to 400 of its particles as dots. Update 1 at 0.0 s: 2000 particles at the start pose. Update 3 at 1.0 s: 501 particles. Update 11 at 4.9 s: odometry 0.107 m off, AMCL 0.041 m off. Last frame, update 21 at 11.3 s: odometry 0.222 m off, AMCL 0.048 m off

Over the whole drive AMCL's estimate is 0.035 m from the truth on average and never more than 0.054 m (notebook). The particle count falls from 2000 to 501 by the third update, the minimum of 500 plus one, because the particles occupy few cells of 0.5 m by 0.5 m by 10°.

A setting that must match the robot. The four `alpha` values say how noisy AMCL should assume odometry is. The bring-up file's 0.2 suits a robot with poor odometry; ours is reliable apart from its 1 percent wheel, so we use 0.005, the value the ROS Navigation Tuning Guide uses for reliable odometry, noting that "when the odometry model is less noisy, the particles are more condensed" (Zheng §5.2). With the file's 0.2 the same run spreads the particles (1471 at the end), and the estimate is up to 0.447 m off, 0.212 m on average (notebook). AMCL's estimate now stays within about 5 cm of the true pose; the next question is where the robot may drive.

5. Where the robot may go: the global and local costmaps

> **Key point:** Both costmaps stack layers on a grid: the map's walls (global only), what the laser sees, and an inflated cost round every obstacle cell, 253 within the robot's radius and falling to 65 at 0.70 m. The global costmap covers the room for the planner and updates once a second; the local one covers 3 m by 3 m round the robot for the controller and updates five times a second.

5.1 Why two costmaps, built from layers

The map of Section 4 has no person, and it marks walls as thin lines of cells that a robot of radius 0.25 m must keep its centre 0.25 m away from. Nav2 answers both with a costmap: a grid in which every cell holds a cost from 0 (free) to 254 (an obstacle in the cell), built by stacking layers, each layer adding one kind of information to the same grid (Nav2 docs, "Environmental Representation"). Here we use Nav2's layers with the bring-up values.

Figure 4 builds the global costmap of our run at 3.0 s, one layer per frame. Watch the person appear in frame 2 as red cells only on the side the laser sees, and then grow a purple ring in frame 3.

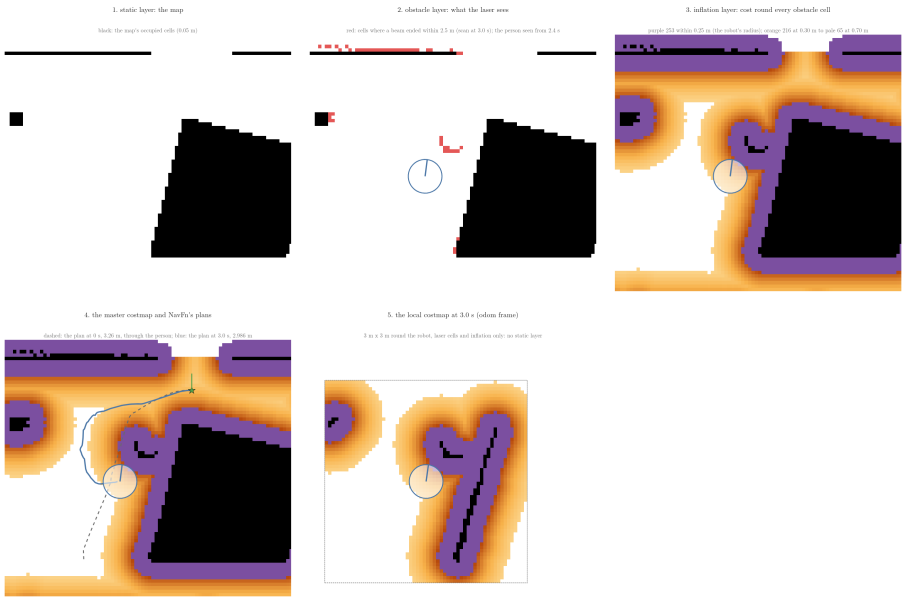


Figure 4: Animation: the global costmap of the main run at 3.0 s, built layer by layer (our Python version of nav2_costmap_2d; not Nav2 running). Frame 1, static layer: the map's occupied cells (black). Frame 2, obstacle layer: cells where a laser beam ended within 2.5 m (red), the person's near side among them. Frame 3, inflation layer: purple, cost 253, within 0.25 m of an obstacle cell; orange 216 at 0.30 m fading to pale 65 at 0.70 m. Frame 4, the master costmap with NavFn's plans: dashed, the plan at 0 s, 3.26 m, through the person; blue, the plan at 3.0 s, 2.986 m, round the person's left. Frame 5, the local costmap at 3.0 s in the odom frame: 3 m by 3 m round the robot, laser cells and inflation only

The three layers, as the bring-up file sets them (`nav2_params.yaml`; plug-ins of `nav2_costmap_2d`):

1. **static layer** (`StaticLayer`): the map's occupied cells become 254;
2. **obstacle layer** (`ObstacleLayer`): each scan clears the cells its beams cross, up to 3.0 m (`raytrace_max_range`), then marks the cells where beams end within 2.5 m (`obstacle_max_range`); a mark stays until a later beam passes through it;
3. **inflation layer** (`InflationLayer`): every cell gets a cost from its distance to the nearest cell of cost 254.

5.2 The inflation cost

The inflation layer uses the rule of the costmap the critics read: 254 on an obstacle cell, 253 within the robot's radius because a robot centred there would overlap the obstacle, then falling exponentially with distance out to `inflation_radius`, 0.70 m, at the rate `cost_scaling_factor`, 3.0 per metre. For a cell 0.30 m from an obstacle, 0.05 m beyond our robot's radius of 0.25 m:

$$252 \times e^{-3.0 \times (0.30 - 0.25)} = 216.9$$

which the code cuts to the whole number 216. The falling costs make the planner prefer cells farther from obstacles: 216 at 0.30 m, 119 at 0.50 m and 65 at 0.70 m (notebook).

5.3 Global and local

The **global costmap** (G-3347) is the one the planner uses: it covers the whole map in the map frame, with all three layers, updated once a second (`update_frequency` 1.0). The **local costmap** (G-3348) is the one the controller uses: a rolling window of 3 m by 3 m centred on the robot in the odometry frame, with the obstacle and inflation layers only, updated five times a second (`update_frequency` 5.0, `rolling_window` true). Both use 0.05 m cells.

The two serve different needs: the planner searches the whole room for a path, while the controller samples commands only over the few metres round the robot, and needs them up to date (Nav2 docs, "Environmental Representation"). The person shows the difference in time (notebook):

- at 2.4 s the laser first sees the person, and the local costmap's update at 2.4 s marks 9 of their cells;
- the global costmap's next update is at 3.0 s, when it marks 11 cells.

So the controller can react to the person 0.6 s before the planner can. Both costmaps are ready; the next section covers the servers that use them in turn.

6. The behaviour tree and the planner server

> **Key point:** The behaviour tree navigator ticks Nav2's default tree every 10 ms; once a second it checks whether the path is still valid (goal unchanged, less than 4 m of path left, no obstacle on it) and asks the planner server for a new one only if it is not. In our run NavFn plans 3.26 m at 0 s, and at 3.0 s, when the global costmap holds the person, 2.986 m round them.

6.1 The behaviour tree navigator

The goal of the Overview arrives as a NavigateToPose action at the **BT navigator** (G-3342), `bt_navigator`: the server that turns a navigation goal into ticks of a behaviour tree, whose nodes call the other servers' actions (Nav2 docs, "Behavior Trees"). The navigator ticks the tree every 10 ms (`bt_loop_duration` 10) until the tree returns success or

failure. The tree is the default file `navigate_to_pose_w_replanning_and_recovery.xml` (`nav2_bt_navigator/behavior_trees`, branch `main`); the tree's nodes are taught with behaviour trees, and here we follow what they do with our goal.

Its main branch is a `PipelineSequence` of selectors (which planner, controller and checkers to use), then two children:

1. a `RateController` at 1 Hz round the path branch: a new path only if one is needed;
2. `FollowPath`, which sends the path to the controller server and stays running while the robot drives.

The path branch first tries a check, `CheckIfNewPathNeeded`, and only if the check fails calls `ComputePathToPose`. The check passes when all of these hold:

- the goal has not been changed (`GlobalUpdatedGoal`, inverted);
- the path left from the robot's nearest pose is shorter than 4.0 m (`IsGoalNearby`, `proximity_threshold`);
- that remaining path is valid: the planner server's IsPathValid service finds no pose on a cell of cost 253 or more in the global costmap (`ValidatePath`; `is_path_valid_service.hpp`).

This check is in the tree on `main`. In the released versions, Humble, Jazzy and Kilted, the 1 Hz branch calls `ComputePathToPose` directly, so the plan is remade every second (the plan remade every second).

A recovery branch runs only when the planner or the controller fails: it clears the costmaps and then asks the **behavior server** (G-3345), `behavior_server`, the server that runs recovery behaviours such as spinning in place, waiting and backing up as actions on the local costmap (Nav2 docs, "Navigation Servers"; bring-up `behavior_server`: `cycle_frequency` 10). The recovery branch never runs in our goal. Our run's ticks of the path branch (notebook):

Time (s)	Path left (m)	Path valid	What the tree does
0.0	none yet		ComputePathToPose: 3.26 m
1.0	3.025	yes	keeps the path
2.0	2.45	yes	keeps the path
3.0	2.0	no, blocked at pose 68	ComputePathToPose: 2.986 m
4.0 to 11.0	2.986 and less	yes	keeps the path

The person stepped onto the route at 2.4 s, but until 3.0 s only the local costmap held them (Section 5.3); the check at 3.0 s was the first to find the path blocked.

6.2 The planner server and NavFn

The **planner server** (G-3343), `planner_server`, answers ComputePathToPose with the plug-in named in the request, on its global costmap (Nav2 docs, "Navigation Servers"). The bring-up's plug-in, `GridBased`, is **NavFn** (G-3346) (`nav2_navfn_planner`) with `use_astar`

false, so it spreads costs like Dijkstra's algorithm (G-2565) rather than A* (G-2577). NavFn works in three steps (`navfn.cpp`):

1. turn each costmap cell into a traversal cost, so that a free cell still costs something and an inflated one costs more;
2. spread a potential, the cheapest cost to reach each cell, outwards from the robot's cell, like a wave whose front cuts across cells instead of following grid lines (the planar-wave update, the idea of the fast marching method (G-2707));
3. walk from the goal down the potential's slope in steps of half a cell, and reverse the walk into the path.

The traversal cost. With:

- c is the costmap cost of a cell, from 0 to 252;
- h is NavFn's traversal cost for the cell.

A cell of costmap cost 216, 0.30 m from an obstacle:

$$h = 50 + 0.8 \times 216 = 222.8$$

The same rule in symbols:

$$h = 50 + 0.8\,c$$

So the cell costs 4.46 times a free cell, which costs 50. Cells of cost 253 and 254 get 254, which the wave never enters. The 50 for a free cell makes the potential grow with distance even in open space, and the factor 0.8 spreads the costs 0 to 252 evenly over the range from 50 to about 252, which keeps paths in the middle of a passage (the NavFn source's comment, quoted in Zheng §2.2).

Our two plans (notebook, Figure 4, frame 4):

- at 0.0 s, 3.26 m and 130 poses, from the robot to the door, through where the person will stand;
- at 3.0 s, 2.986 m, from where the robot then was, round the person's left; NavFn's wave reached 7358 of the 16000 cells.

The plan at 0 s is shorter than the 3.414 m of A* on 0.5 m cells, because NavFn's 0.05 m cells and its wave cut corners that 0.5 m cells could not. A path is not yet a command; the controller server turns it into commands.

7. The controller server

> **Key point:** The controller server runs 20 times a second: it puts the path in the odometry frame, keeps the part ahead of the robot, checks whether the goal is reached or progress has stopped, and asks its plug-in for a command. Our run uses DWB, the controller Nav2's documentation calls the default; the bring-up file itself names MPPI.

7.1 One controller cycle

The path of Section 6 is in the map frame, but the local costmap is in the odometry frame and moves with the robot. The **controller server** (G-3344), `controller_server`, bridges them in each cycle, 20 times a second (`controller_frequency` 20.0) (`nav2_controller/src/controller_server.cpp`):

1. transform the path into the odometry frame with AMCL's correction;
2. keep the part from the pose nearest the robot to 2.0 m ahead, and inside the local costmap (`FeasiblePathHandler`, `prune_distance`);
3. check the goal: within 0.25 m, then within 0.25 rad of its heading (`SimpleGoalChecker`, stateful);
4. check progress: the robot must move 0.5 m every 10 s (`SimpleProgressChecker`);
5. ask the plug-in `FollowPath` for a command and publish it on `cmd_vel_nav`.

The goal checker and progress checker are taught with recovery behaviours. Steps 1 to 4 and our plug-in, DWB, come from the Python version of the controller server written for DWA and DWB.

7.2 DWB in our run

DWB (G-3255) samples speed and turn-rate pairs the robot can reach, rolls each out for 1.7 s, scores the rollouts with its critics and keeps the cheapest. At 1.0 s in our run (notebook):

- the window of speeds is 0 to 0.6 m/s and of turn rates -1 to 1 rad/s;
- of 375 sampled pairs, 373 pass every critic;
- the best is 0.5684 m/s and 0.2632 rad/s, which the controller publishes on `cmd_vel_nav`.

7.3 Which controller: DWB, MPPI, regulated pure pursuit, TEB

Nav2's documentation says "The DWB controller is the default controller" (Nav2 docs, DWB controller page), but the bring-up file on `main` gives `FollowPath` the plug-in `nav2_mppi_controller::MPPIController`. We run DWB because its algorithm is the one taught in this chapter; how DWA reached ROS 1 and then Nav2 is told with DWA and DWB. The controller plug-ins named in Nav2's list (Nav2 docs, "Navigation Plugins"):

- **MPPI controller** (G-3354), `nav2_mppi_controller`: it adds random Gaussian changes to the last best command sequence, simulates the noisy sequences, scores them with critics and combines them with a softmax weighting (its README); the bring-up runs 2000 sequences of 56 steps of 0.05 s, a 2.8 s horizon, and the README measures "50+ Hz on a modest Intel processor"; MPPI itself is the subject of a later chapter on sampling-based control;
- regulated pure pursuit (G-3154), `nav2_regulated_pure_pursuit_controller`: pure pursuit that slows on tight curves and near obstacles, measured at "well over 1 kHz" (its README);
- TEB: not part of the `navigation2` repository but the separate package `teb_local_planner` (GitHub `rst-tu-dortmund`), listed among Nav2's plug-ins.

Whichever plug-in computes the command, two further nodes act on it before it reaches the wheels.

8. After the controller: the velocity smoother and the collision monitor

> **Key point:** The velocity smoother limits how fast each command may change, 20 times a second; the collision monitor then checks the smoothed command against the raw scan, outside every planner, and scales it by the time to collision over 1.2 s. In our run the smoother turns DWB's first 0.4737 m/s into 0.025 m/s, and the monitor acts once, at 2.55 s, scaling the command by 0.917.

8.1 The velocity smoother

DWB's first command, at 0.0 s, is 0.4737 m/s and 0.6842 rad/s, while the robot is at rest. A base given that command would change its speed in one step, far beyond its acceleration limit. The velocity smoother (G-3085), `velocity_smoother`, runs 20 times a second (`smoothing_frequency` 20.0) and, in open loop (`feedback: OPEN_LOOP`), changes its last output towards the new command by at most the acceleration limit over one period (`velocity_smoother.cpp`, `applyConstraints`). With:

- v and ω are the smoother's last output speed and turn rate, in m/s and rad/s, both 0 before the first command;
- v_{new} and ω_{new} are its next output, in m/s and rad/s, moved towards DWB's command;
- a is the speed's acceleration limit, 0.5 m/s^2 for our robot, and α the turn rate's, 3.2 rad/s^2;
- f is the smoothing frequency, 20 Hz.

The first output, from rest:

$$v_{\text{new}} = 0 + 0.5/20 = 0.025 \text{ m/s}$$

$$\omega_{\text{new}} = 0 + 3.2/20 = 0.16 \text{ rad/s}$$

In symbols, each period the output moves towards DWB's command by at most:

$$|v_{\text{new}} - v| \leq a/f$$

$$|\omega_{\text{new}} - \omega| \leq \alpha/f$$

At 1.0 s, DWB asks for 0.5684 m/s and the smoother gives 0.525 m/s, its last output of 0.5 m/s plus 0.025 (notebook). The smoothed command goes out on `cmd_vel_smoothed`.

8.2 The collision monitor

The costmaps of Section 5 update at 1 Hz and 5 Hz, and DWB plans on what they held at their last update. The **collision monitor** (G-3350), `collision_monitor`, is a last check that works on the raw scan at every command: it looks for scan returns inside zones round the robot and slows or stops the command, "bypassing navigation's planners" (Macenski et al. 2023 §IX; `nav2_collision_monitor` README). The bring-up file enables one zone, `FootprintApproach`.

The approach zone works on time (`polygon.cpp`, `getCollisionTime`):

1. move the robot's footprint (16 points on a circle of the robot's radius) forward with the command, 0.1 s at a time (`simulation_time_step`), up to 1.2 s (`time_before_collision`);
2. at the first step whose footprint holds 6 or more scan returns (`min_points`), take the loop's time at that step as the time to collision;
3. scale the command by the time to collision over 1.2 s.

Keeping the robot that many seconds from any collision is the **approach zone** (G-3351) (`nav2_collision_monitor` README: "keep the robot always M seconds from any collision").

Figure 5 shows the one time it acts in our run, at 2.55 s, 0.865 m from the person's centre. Watch the footprint step forward until, at the twelfth step, 6 of the person's returns fall inside it.

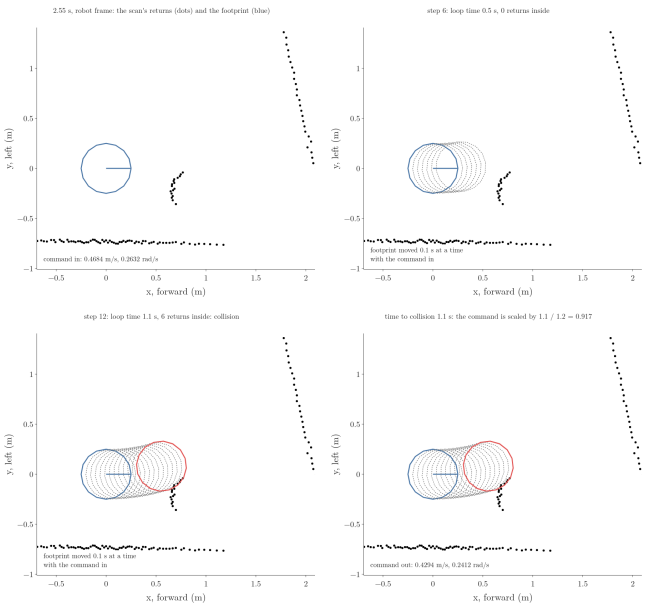


Figure 5: Animation: the collision monitor's approach zone at 2.55 s of the main run, in the robot's frame, x forward (our Python version; not Nav2 running). Dots: the scan's returns, the person's arc ahead and to the right. Blue: the footprint, 16 points on a circle of 0.25 m. Each frame moves it forward 0.1 s more with the command in, 0.4684 m/s and 0.2632 rad/s. Frame 'step 12: loop time 1.1 s, 6 returns inside: collision' (red footprint). Last frame: time to collision 1.1 s, so the command is scaled by 1.1 / 1.2 = 0.917, to 0.4294 m/s and 0.2412 rad/s

The scaling at 2.55 s, with:

- t_c is the time to collision, 1.1 s;
- T is `time_before_collision`, 1.2 s;
- v and ω are the command in, 0.4684 m/s and 0.26316 rad/s; v' and ω' the command out.

$$1.1/1.2 = 0.9167$$

$$0.4684 \times 0.9167 = 0.4294 \text{ m/s}$$

$$0.26316 \times 0.9167 = 0.2412 \text{ rad/s}$$

The same steps in symbols:

$$v' = v\, t_c/T$$

$$\omega' = \omega\, t_c / T$$

The loop reports the time at the start of its step, 1.1 s, although the footprint has by then moved 12 steps of 0.1 s; a collision at the very first step would give 0 and a full stop. In the next cycle DWB, which has seen the person in the local costmap since 2.4 s, already steers away, and the monitor never acts again; the robot's disk passes the person's at 0.039 m, at 4.25 s (notebook).

8.3 Stop, slowdown and limit zones

The approach zone is one of four kinds. The example parameter file of the package `nav2_collision_monitor`, named `collision_monitor_params.yaml`, defines all of them; zones of the first three kinds act on returns inside a fixed polygon:

- **stop**: 4 or more returns in a box 0.3 m ahead of the robot's centre and 0.6 m wide set the command to 0;
- **slowdown**: returns in a box 0.8 m by 0.8 m multiply the command by 0.3;
- **limit**: returns in a box 1.0 m by 1.0 m cap the speed at 0.4 m/s and the turn rate at 0.5 rad/s;
- **approach**: as above, here with 2.0 s.

Each zone's polygon, its action and its number of returns make a **collision zone** (G-3352) (`nav2_collision_monitor` README). With several zones, the monitor checks them in order and keeps the safest command, smallest in speed and turn rate together; a stop ends the checks (`collision_monitor_node.cpp`, `process`).

A pitfall: zones are sized for one robot. We ran the same goal with all four example zones on our robot (notebook). Figure 6 shows what happened at 3.70 s: the person's returns fell into the stop box as DWB passed them, the command became 0, and since the person never moves, the robot stayed there until the progress checker failed the goal at 12.45 s.

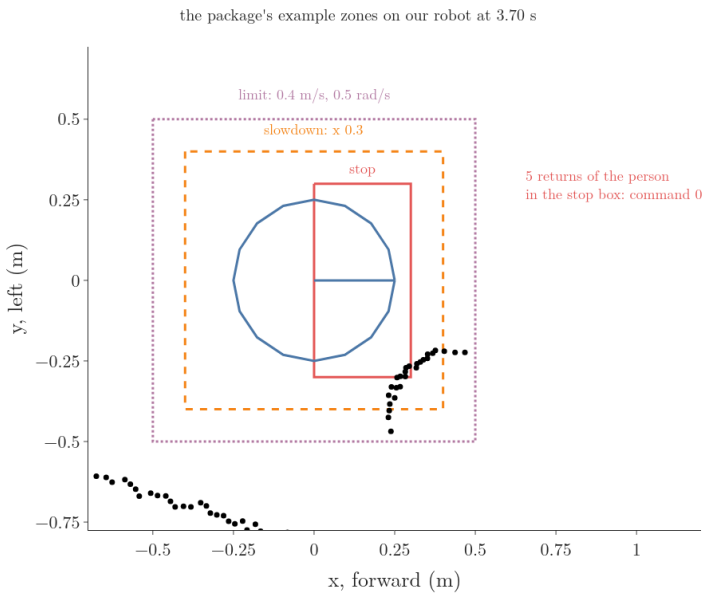


Figure 6: The collision monitor package's example zones on our robot at 3.70 s of the example-zones run (our Python version; not Nav2 running), robot frame, x forward. Red: the stop box, 0.3 m ahead and 0.6 m wide; orange dashed: the slowdown box, 0.8 m by 0.8 m; purple dotted: the limit box, 1.0 m by 1.0 m; blue: the footprint, radius 0.25 m. Black dots: the scan's returns; 5 of the person's fall inside the stop box, so the command is 0

The example's stop box is square and our robot is round: the box's front corners lie 0.424 m from the robot's centre, 0.174 m beyond its edge, so a corner caught the person's edge while the two disks were still 0.149 m apart (notebook). Zones must be drawn round the robot's own footprint and agree with how close its controller is allowed to pass; the bring-up enables only the approach zone. The command that leaves the monitor is the `cmd_vel` the base obeys, and every server on the goal's route has now been described.

9. The goal from start to finish

> **Key point:** With our Python version of every server running, the goal of the Overview takes 11.50 s, against 9.45 s in the clear room and 8.0 s for the timed elastic band of trajectory optimisation: the person delays the robot by 2.05 s, and DWB with a 1 rad/s turn limit is slower than the optimiser.

Sections 2 to 8 examined the servers one at a time. Figure 7 plays the whole goal in the room. Watch the blue plan change at 3.0 s and the orange rollout bend round the person before that, from 2.4 s.

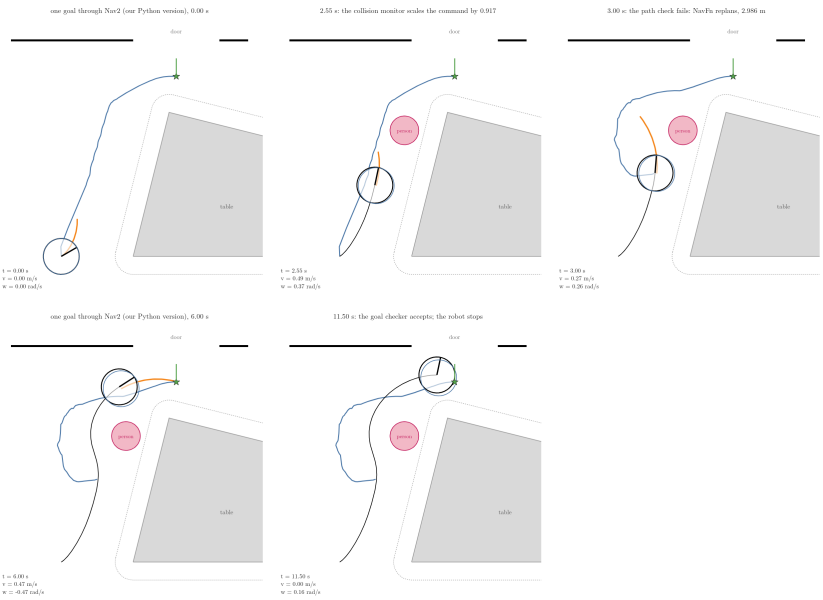


Figure 7: Animation: the whole goal in the room, one frame every 0.25 s of the main run (our Python version of Nav2's servers; not Nav2 running). Black disk: the robot where it truly is, with its track; blue ring: where AMCL says it is; blue line: NavFn's plan; orange: the DWB rollout chosen in that cycle; pink: the person, from 2.4 s; green star: the door pose. Frames held at 2.40 s (the laser sees the person), 2.55 s (the collision monitor scales the command by 0.917) and 3.00 s (the path check fails and NavFn replans, 2.986 m). Last frame, 11.50 s: the goal checker accepts and the robot stops

The goal, server by server (notebook):

Time (s)	Server	What happens
before 0	lifecycle manager	17 nodes configured, then activated
0.0	`bt_navigator`	goal received; tree ticked every 10 ms
0.0	`planner_server`	NavFn: 3.26 m
0.0	`controller_server`, `velocity_smoother`	DWB 0.4737 m/s, smoothed to 0.025 m/s
2.4	local costmap	the person's first 9 cells
2.55	`collision_monitor`	command scaled by 0.917
3.0	global costmap, `bt_navigator`	the person's 11 cells; path check fails
3.0	`planner_server`	NavFn: 2.986 m
4.25		closest pass: 0.039 m between the disks
11.50	`controller_server`	goal checker accepts

Time (s)	Server	What happens
11.51	`bt_navigator`	the tree succeeds; the result goes back

The goal checker accepts AMCL's estimate within 0.230 m of the door pose; the robot truly stops 0.267 m and 12.0° from it, inside the 0.25 m and 0.25 rad tolerances plus AMCL's few centimetres of error (notebook). The top speed is 0.593 m/s, close to the wheels' 0.6 m/s.

The times, from the same start to the same goal:

- our Python version of Nav2 with DWB, the person at 2.4 s: 11.50 s;
- the same, in the clear room: 9.45 s;
- the timed elastic band of trajectory optimisation, with the person: 8.0 s, and 6.7 s in the clear room (the receding-horizon loop).

The person delays our Nav2 run by 2.05 s and the timed elastic band by 1.3 s. Why DWB is slower than TEB, with one command per rollout and a turn on the spot at the end, is worked out in DWA and DWB, whose own DWB run with the plan remade every second arrives at 13.3 s. That run follows the A* route on 0.5 m cells, with costmaps built from the exact room; ours follows NavFn's path on 0.05 m cells, with costmaps built from the laser, and passes the smoother and the monitor, so the numbers differ. The final approach is the same: NavFn's path also reaches the door pose heading 7°, so our robot stops moving forward at 8.35 s facing 4.9° and turns on the spot for 3.15 s, to 78.0° (notebook). Nav2 itself only orders the servers and routes the commands. ROS 1 arranged these same pieces differently.

10. ROS 1 `move_base` compared

> **Key point:** ROS 1 ran the same pieces (a global planner, a local planner, two costmaps, recoveries) inside one node, `move_base`, ordered by a fixed state machine; Nav2 splits them into lifecycle servers and orders them with a behaviour tree that can be changed without changing code.

Every server of Sections 4 to 8 has an ancestor in ROS 1. The original navigation stack, written at Willow Garage for the PR2 robot, was "based around an unconfigurable state machine, `move_base`" (Macenski et al. 2023 §I). **`move_base`** (G-3353) is the ROS 1 node that holds a global planner and a local planner as plug-ins, a global and a local costmap, and a list of recovery behaviours, and switches between three states, PLANNING, CONTROLLING and CLEARING (`move_base.h`, `ros-planning/navigation`, branch `noetic-devel`). Figure 8 sets the two side by side.

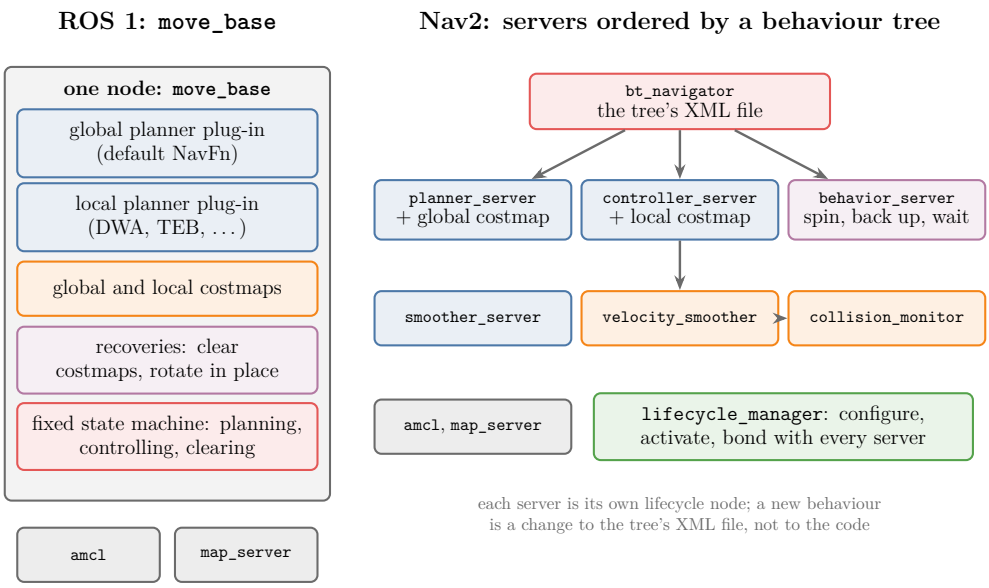


Figure 8: ROS 1 move_base against Nav2. Left: one node, move_base, holding a global planner plug-in (default NavFn), a local planner plug-in (DWA, TEB and others), the global and local costmaps, recoveries (clear the costmaps, rotate in place) and a fixed state machine (planning, controlling, clearing); amcl and map_server run beside it. Right: Nav2's separate servers: bt_navigator with the tree's XML file calls planner_server with the global costmap, controller_server with the local costmap and behavior_server (spin, back up, wait); the commands pass velocity_smoother and collision_monitor; smoother_server, amcl and map_server run beside them; the lifecycle manager configures, activates and bonds with every server

The differences, from `move_base.cpp` and the Nav2 files of this Note:

	ROS 1 `move_base`	Nav2
Programs	one node	one server per job, plus `amcl`, `map_server`
Order of steps	fixed state machine in C++	behaviour tree in an XML file
Replanning	`planner_frequency` 0.0: plan once, again only on failure	the tree's 1 Hz check, a new path when needed
Recoveries	clear the costmaps beyond 3.0 m, rotate, clear them beyond 4 robot radii, rotate	clear the costmaps, spin, wait, back up, in a round robin
Start-up	no lifecycle states	lifecycle manager: configure, activate, bonds
Safety on the command	none in the node	velocity smoother and collision monitor

Both run the controller 20 times a second (`controller_frequency` 20.0). The ROS Navigation Tuning Guide shows what a fixed state machine cost: when a ROS 1 robot had used all its recoveries, `move_base` aborted the goal, so the guide's authors wrote their own state machine round `move_base` to keep trying (Zheng §6); in Nav2 the same change is an edit to the tree's XML file. The remaining parts of the Nav2 ecosystem are the tools for testing all of this before a robot exists.

11. Simulators and tools

> **Key point:** Nav2 is developed and tested in simulators, mainly Gazebo, before it runs on a robot; research on learned navigation adds simulators built for speed, photographs of real buildings, drones or crowds.

Our run replaced the robot and its laser with Python. Real development does the same with a simulator that publishes the same topics a robot would, so that the Nav2 of Sections 2 to 9 runs unchanged. How a physics simulator steps and which engine to choose is taught with physics simulators; here are the ones named for navigation, each from its own page:

Simulator	What it simulates	Used for
Gazebo	robots with physics, rendering and sensors, fully integrated with ROS; Nav2 ships simulations of a robot in a sandbox and a warehouse	testing Nav2 before hardware (Macenski et al. 2023 §IX; Gazebo Sim README)
Nav2's loopback simulator	odometry computed from `cmd_vel` on a "frictionless plane", no physics, with the map → odom and odom → base_link transforms	testing higher-level behaviours without concern for localisation accuracy or dynamics (`nav2_loopback_sim` README); the bring-up file also gives it scan settings, which our laser copies
Isaac Lab	GPU-accelerated physics and sensors (cameras, lidar) on NVIDIA Isaac Sim	reinforcement learning, imitation learning and motion planning (Isaac Lab README)
Habitat-Sim	3D scans and models of real indoor spaces, with configurable cameras and robots	embodied navigation research (Habitat-Sim README)
Flightmare	quadrotors, with a Unity renderer and a separate physics engine	drone planning and reinforcement learning, hundreds of drones in parallel (Flightmare README)

Simulator	What it simulates	Used for
CrowdNav	a robot among simulated walking people	learning to navigate through crowds (CrowdNav README)

Around the simulator are the tools a Nav2 user works with: RViz (G-3230) with Nav2's panel, which sends goals and shows the lifecycle states and the action feedback, and the Simple Commander, a Python interface for sending goals from a script (Macenski et al. 2023 §IX). The learned navigation policies of the next chapter are trained in simulators of the second kind, Habitat among them.

12. Summary

Server	Its job	In our Python run
lifecycle manager	configure, then activate, 17 nodes in order; bonds	34 transitions; heartbeat 0.25 s, 4 s timeout
`map_server`, AMCL	the map; the robot's pose on it, as a map → odom correction	odometry 0.222 m off; AMCL at most 0.054 m
global and local costmaps	static, obstacle and inflation layers	253 within 0.25 m, 216 at 0.30 m, 65 at 0.70 m
`bt_navigator`	ticks the tree; replans only when the path is bad	path check fails at 3.0 s
`planner_server` (NavFn)	Dijkstra-like wave on the global costmap	3.26 m, then 2.986 m
`controller_server` (DWB)	20 Hz: prune, check goal and progress, command	0.5684 m/s at 1.0 s
velocity smoother	limit each change to the acceleration limits	0.4737 m/s asked, 0.025 m/s sent at 0 s
collision monitor	slow or stop on raw scan returns, outside the planners	× 0.917 at 2.55 s

- One goal crosses the whole of Nav2 and the robot reaches the door in 11.50 s, replanning once round the person at 3.0 s, because each server does one job in order: AMCL places the robot, the costmaps place the obstacles, the tree decides when to plan, NavFn plans, DWB commands, and the smoother and the monitor check each command; this is our Python version of Nav2's servers, written from their code and bring-up parameters.
- The **lifecycle manager** (G-3341) configures every node before activating any, so no server meets a message from a server that is not ready, and its bonds bring everything down if one stops.
- **AMCL** (G-3349) holds the estimate within 0.054 m while odometry drifts 0.222 m, because the laser's likelihood field corrects the particles; its odometry noise settings must match the robot, or the particles spread (0.447 m off with the bring-up's 0.2).
- The **global costmap** (G-3347) updates once a second for the planner and the **local costmap** (G-3348) five times a second for the controller, so the controller sees the person 0.6 s before the planner does.
- The **BT navigator** (G-3342) asks the **planner server** (G-3343) for a path only when the 1 Hz check fails, so **NavFn** (G-3346) runs twice in our goal, not once a second.
- The **controller server** (G-3344) turns the map-frame path into 20 commands a second through its plug-in; DWB in our run, the **MPPI controller** (G-3354) in the bring-up file.
- The velocity smoother keeps the commands within the acceleration limits, so the base never receives a command it cannot follow.
- The **collision monitor** (G-3350) acts on the raw scan at every command, so a person not yet in the costmaps still slows the robot; its **approach zone** (G-3351) scales the

command by the time to collision over 1.2 s, and its **collision zones** (G-3352) must be sized to the robot's own footprint, or they stop it beside an obstacle permanently.
- ROS 1's `move_base` (G-3353) held the same pieces in one node with a fixed state machine, so changing the order of steps meant changing code; Nav2's tree is a file.

The robot can now navigate a mapped room with a real software stack, end to end. One limitation remains: Nav2 drives to a goal pose it is given, while a task such as putting a cup on a spot where a bottle stands needs a list of actions, each with its own places and motions; task and motion planning finds them.

13. Sources

Built from

- Robotics Back-End, "ROS2 Nav2 - Navigation Stack in 1 Hour [Crash Course]", YouTube, https://www.youtube.com/watch?v=idQb2pB-h2Q (a first run: the map and its resolution, the 2D pose estimate, a goal with the global planner and the controller in RViz, an aborted goal)
- Nav2 documentation, "Navigation Concepts": Navigation Servers, Environmental Representation, Behavior Trees, State Estimation; the DWB controller page; "Navigation Plugins". Source `ros-navigation/docs.nav2.org`, https://docs.nav2.org/ (Nav2 docs)
- Nav2 source, `ros-navigation/navigation2`, branch `main`, read 2026-10-12, https://github.com/ros-navigation/navigation2:
  - `nav2_bringup`: the launch files `bringup_launch.py`, `localization_launch.py`, `keepout_zone_launch.py`, `speed_zone_launch.py` and `navigation_launch.py`, and the parameter file `nav2_params.yaml`;
  - `nav2_lifecycle_manager`: `lifecycle_manager.cpp`;
  - `navigate_to_pose_w_replanning_and_recovery.xml`, the default tree, in the folder `behavior_trees` of `nav2_bt_navigator`;
  - `nav2_behavior_tree` plug-ins: `pipeline_sequence.cpp`, `rate_controller.cpp`, `is_goal_nearby_condition.cpp`, `validate_path_action.cpp`;
  - `nav2_planner`: `planner_server.cpp`, `is_path_valid_service.hpp`;
  - `nav2_navfn_planner`: `navfn.cpp`, `navfn_planner.cpp`;
  - `nav2_costmap_2d`: the layers `inflation_layer.hpp` and `obstacle_layer.cpp`, and the node `costmap_2d_ros.cpp`;
  - `nav2_controller`: `controller_server.cpp`;
  - `nav2_velocity_smoother`: `velocity_smoother.cpp`;
  - `nav2_collision_monitor`: `polygon.cpp`, `collision_monitor_node.cpp`, `kinematics.cpp`, `README.md`, `collision_monitor_params.yaml`;
  - `nav2_amcl`: `amcl_node.cpp` (the node), `likelihood_field_model.cpp` (the laser model), `differential_motion_model.cpp` (the motion model) and `pf.c` (the filter);
  - `nav2_mppi_controller`, `nav2_regulated_pure_pursuit_controller` and `nav2_loopback_sim`: `README.md`.
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A. and Ferguson, M. (2023). "From

the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2". *Robotics and Autonomous Systems*. §I (move_base, behaviour trees), §IX (lifecycle manager, collision monitor, velocity smoother, simulation). https://arxiv.org/abs/2307.15236 (Macenski et al. 2023)
- Zheng, K. (2017). "ROS Navigation Tuning Guide". §2.2 (NavFn's costs), §5.2 (AMCL's motion and measurement parameters), §6 (recovery behaviours). https://arxiv.org/abs/1706.09068 (Zheng)
- ROS 1 navigation source, `ros-planning/navigation`, branch `noetic-devel`: `move_base/include/move_base/move_base.h`, `move_base/src/move_base.cpp`, read 2026-10-12. https://github.com/ros-planning/navigation

Other references

- Gazebo Sim README, `gazebosim/gz-sim`; Isaac Lab README, `isaac-sim/IsaacLab`; Habitat-Sim README, `facebookresearch/habitat-sim`; Flightmare README, `uzh-rpg/flightmare`; CrowdNav README, `vita-epfl/CrowdNav`; all on GitHub, read 2026-10-12.
- The Python version of DWB written for DWA and DWB (RO-078, `images/dwbkit.py`), copied into this Note's `images/`.

14. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Nav2 (G-3340)	The navigation stack of ROS 2: separate programs (a lifecycle manager, AMCL, costmaps, a behaviour tree navigator, planner, controller and behaviour servers, a velocity smoother and a collision monitor) that together take a goal pose and drive a robot there.
Lifecycle manager (Nav2) (G-3341)	The node that configures a list of lifecycle nodes in order, then activates them in order, and keeps a heartbeat (bond) with each; it guarantees every server is ready before the first goal and brings the system down if one fails.
Map server (G-3355)	The Nav2 node that loads a saved occupancy grid map and serves it on the `/map` topic, so AMCL and the global costmap work on the same map.

Term	Meaning
AMCL (adaptive Monte Carlo localisation) (G-3349)	Nav2's localiser: a particle filter on a known map that moves its particles with odometry, weights them with a laser likelihood field and adapts their number; it publishes the map-to-odometry transform that corrects odometry's drift.
Global costmap (G-3347)	The costmap the planner uses: the whole map in the map frame, with static, obstacle and inflation layers, updated about once a second; it tells the planner where the robot's centre may go.
Local costmap (G-3348)	The costmap the controller uses: a small rolling window centred on the robot (3 m by 3 m in Nav2's bring-up) in the odometry frame, built from the sensors and inflated, updated several times a second so new obstacles appear quickly.
BT navigator (G-3342)	Nav2's server that receives a navigation goal as an action and ticks a behaviour tree, by default every 10 ms, whose nodes call the planner, controller and behaviour servers; changing the tree's XML file changes how the robot navigates.
Behavior server (G-3345)	Nav2's server that runs recovery behaviours such as spinning in place, waiting and backing up as actions on the local costmap, when the behaviour tree's recovery branch asks for them.
Planner server (G-3343)	Nav2's server that answers path requests (ComputePathToPose) with a planner plug-in such as NavFn, on the global costmap it hosts, and checks whether a path is still free (IsPathValid).
NavFn (G-3346)	Nav2's default global planner: it gives each costmap cell a traversal cost $50 + 0.8c$, spreads a potential from the robot's cell like a wave (Dijkstra's order, or A*'s), and follows the potential's slope from the goal in half-cell steps to make the path.

Term	Meaning
Controller server (G-3344)	Nav2's server that follows a path (FollowPath): 20 times a second by default it transforms the path into the odometry frame, keeps the part ahead, checks goal and progress, and asks a controller plug-in such as DWB or MPPI for a velocity command.
MPPI controller (Nav2) (G-3354)	The controller plug-in in Nav2's bring-up file: it adds random Gaussian changes to its last best command sequence, simulates the noisy sequences, scores them with critics and blends them with softmax weights into the next command.
Collision monitor (G-3350)	Nav2's node between the velocity smoother and the robot base that checks each velocity command against raw sensor returns in zones round the robot and stops, slows or limits it, independently of the planners and costmaps.
Approach zone (collision monitor) (G-3351)	The collision monitor's model that moves the robot's footprint forward with the command in short steps and, if returns enter it within a set time (1.2 s in Nav2's bring-up), scales the command by the time to collision over that time, so the robot always stays that long from a collision.
Collision zone (stop, slowdown, limit) (G-3352)	A polygon round the robot with an action and a minimum number of sensor returns: enough returns inside it stop the robot, multiply its command by a ratio, or cap its speed and turn rate; zones must be sized to the robot's own footprint.
move_base (G-3353)	The ROS 1 navigation node that held a global planner, a local planner, two costmaps and recovery behaviours in one program, switching between planning, controlling and clearing with a fixed state machine; Nav2 replaced it.

Term	Meaning
Dijkstra's algorithm (G-2565)	A search that always settles the vertex with the smallest distance from the start found so far, which is then final, and offers each of its neighbours the route through it, keeping the shorter; with every edge cost at least 0 it finds the shortest route to every vertex (3.414 m to the door on the room grid with diagonals).
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
Particle filter (G-2492)	A Bayes filter that stores the belief as a set of weighted samples (particles) and runs each round as move, weigh, resample; it can hold beliefs of any shape, such as a belief with three modes, one at each of three doors.
DWB controller (G-3255)	Nav2's controller that, 20 times a second by default, samples speed and turn-rate commands within the acceleration limits, rolls each out, scores the rollouts with plug-in critics and sends the lowest; the successor of ROS 1's DWA, it follows the global plan and cannot search round an obstacle on its own.
ROS 2 (Robot Operating System 2) (G-3221)	An open-source set of libraries and tools that runs a robot's software as separate programs (nodes) passing typed messages over topics, services and actions, and that configures, starts, records and shows them; it lets parts written by different people work together at their own rates.
Autonomy stack (G-3210)	The whole of a robot's software arranged in layers, sensing at the bottom and mission, behaviour, motion and control above it, each layer running at its own rate and working from the latest output of the layer above; the slow layers plan far ahead while the fast ones react, so a new reading reaches the wheels in one fast cycle.

Term	Meaning
Lifecycle node (managed node) (G-3236)	A node with the states Unconfigured, Inactive, Active and Finalized that it changes only when another program requests a transition (configure, activate, deactivate, cleanup, shutdown); a manager can then set nodes up and start them in order, and restart or replace them while the system runs.
Launch file (G-3235)	A file, in Python, XML or YAML, that lists the nodes to start with their parameters, names and arguments, and that `ros2 launch` runs as a whole; one command starts the whole stack and one Ctrl+C stops it, and launch files can include others.
Action (ROS 2) (G-3227)	A long job between two nodes, defined in an `.action` file: the client sends a goal, the server sends feedback while it works and a result when it ends, and the client can cancel, such as driving to a door pose; it is built from three services and two topics.
Topic (G-3223)	A named channel of messages, such as `/scan`, that any node can publish to or subscribe to; every subscriber receives a copy of each message and the publisher does not wait, so it carries streams such as scans and commands between nodes running at different rates.
Service (ROS 2) (G-3226)	A request and one reply between two nodes, defined in a `.srv` file, such as asking the map server for the map: the client sends the request and waits, the server computes and replies; it suits short questions, not long jobs.
Wheel odometry (G-2321)	Working out a robot's pose by adding up the wheel motion its encoders measure: ticks to wheel distances, to the robot's move and turn, to a new pose; accurate over short distances but it drifts, because every error stays in the sum.
Occupancy grid (G-2340)	A grid map that stores in each cell a number between 0 and 1, such as the probability that the cell is occupied, instead of a hard 0 or 1; the standard grid map of robotics, since sensor readings are uncertain.

Term	Meaning
Odometry motion model (G-2323)	A motion model that splits the pose change reported by odometry into a first turn, a straight drive and a second turn, and adds independent noise to each, with a standard deviation that grows with the motion; used when wheel encoders exist.
Likelihood field model (G-2391)	A measurement model for range finders that scores each reading by how close its beam endpoint lands to an obstacle in the map (a Gaussian of that distance plus a small uniform part p_{rand}, so no reading scores 0); it is fast and smooth because it skips ray casting, but it ignores what lies along the beam.
Fast marching method (G-2707)	Dijkstra's loop run on a grid with an update that takes a cell's arrival time from two neighbours at once, one across and one up or down, so a front spreading from the goal can cross the grid at any angle; its distances get closer to the true ones as the cells shrink, unlike 8-connected Dijkstra's.
Regulated pure pursuit (G-3154)	Pure pursuit with rules that lower the speed when the commanded arc is tighter than a set radius, to $v_{\max}R/r_{\min}$, and when an obstacle is near, as in Nav2; with a velocity-scaled look-ahead, slowing also shortens the look-ahead, so the car cuts tight turns less.
Velocity smoother (G-3085)	A node between the planner and the robot's base that clamps each velocity command to the speed limits, limits its rate of change and applies a deadband, such as Nav2's `nav2_velocity_smoother`; it turns abruptly changing planner output into commands the drive can follow.
RViz (G-3230)	ROS's visualization tool: it subscribes to topics such as the map, the scans, the plans and the robot's model and draws them together in one 3D view, so we can see what the robot sees and plans.